

DEPI Grid — Technical Documentation

Renewable Telemetry, AI Diagnostics & Chaos Engineering Platform

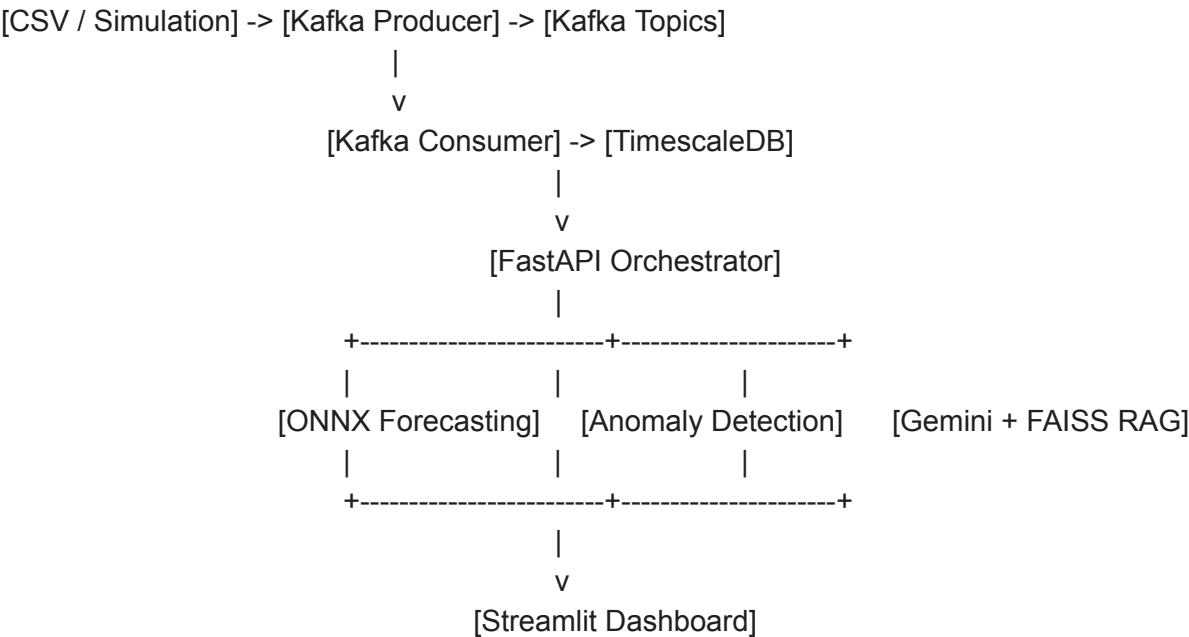
1. Project Overview

DEPI Grid is an end-to-end renewable-energy monitoring platform that combines real-time telemetry streaming, machine-learning forecasting, explainable anomaly detection, and Retrieval-Augmented Generation (RAG) for autonomous repair blueprints.

The system orchestrates three data sources — Wind, Solar, and Weather telemetry — into a unified pipeline:

- **Simulation / Kafka streaming** ingests synthetic sensor data.
- **TimescaleDB** stores time-series telemetry.
- **FastAPI orchestrator** exposes REST endpoints for forecasts, live data, and diagnostics.
- **ONNX + scikit-learn inference engine** performs power forecasting and anomaly detection.
- **Google Gemini + FAISS vector store** generates repair blueprints via RAG.
- **Streamlit dashboard** visualizes grid balance, AI forecasts, and explainable diagnostics in real time.
- **Chaos engineering endpoint** injects synthetic anomalies to validate the diagnostic loop.

Architecture at a Glance



2. File-by-File Technical Breakdown

2.1 sender.py — Tri-Stream Kafka Producer

Purpose: Reads three simulation CSVs (Wind, Solar, Weather) in parallel threads and streams each row to its corresponding Kafka topic with a live UTC timestamp.

Constants

Constant	Description
KAFKA_BROKER	Bootstrap server (localhost:9092)
WIND_TOPIC / SOLAR_TOPIC / WEATHER_TOPIC	Kafka topic names
WIND_CSV / SOLAR_CSV / WEATHER_CSV	Source dataset paths
SIMULATION_SPEED	Sleep duration (seconds) between row emissions; controls streaming cadence

Functions

- **delivery_report(err, msg)**
 - **Purpose:** Kafka producer callback invoked when a message is acknowledged or fails.
 - **Parameters:** err (Kafka error object), msg (The message object that was produced).
 - **Behavior:** Logs delivery failures only.
- **stream_data(csv_path, topic, stream_name, key_field)**
 - **Purpose:** Streams a single CSV file to a single Kafka topic indefinitely.
 - **Parameters:** csv_path (Path to the simulation CSV), topic (Target Kafka topic), stream_name (Human-readable label used in logs), key_field (CSV column used as the Kafka partition key).
 - **Behavior:** Loads CSV via pandas. Creates a `confluent_kafka.Producer`. Iterates rows; injects a live UTC ts field into the payload. Serializes payload to JSON and produces it with the partition key. Loops infinitely when CSV is exhausted.
- **start_multi_sender()**
 - **Purpose:** Launches three daemon threads — one per telemetry stream — enabling concurrent tri-stream emission.
 - **Behavior:** Joins threads so the main process waits for them.

2.2 receiver.py — Kafka Consumer -> TimescaleDB Writer

Purpose: Subscribes to all three telemetry topics, buffers records in memory, and bulk-inserts

them into TimescaleDB in batches.

Constants

Constant	Description
KAFKA_BROKER	Bootstrap server
GROUP_ID	depi_db_writer — consumer group identifier
DB_*	TimescaleDB connection credentials
BATCH_SIZE	Number of records buffered per topic before flush

Functions

- **start_receiver()**
 - **Purpose:** Main entry point for the consumer process.
 - **Behavior:** Creates a Kafka Consumer with `auto.offset.reset='earliest'` and manual commits. Subscribes to telemetry topics. Opens a persistent `psycpg2` connection. On each `consumer.poll()`, parses the payload, falls back to UTC time if missing, and routes the record to a buffer. When a buffer reaches `BATCH_SIZE`, it issues a parameterized `INSERT` via `execute_values`, commits the DB transaction, commits the Kafka offset synchronously (at-least-once semantics), and clears the buffer.

2.3 simulation.py — Direct-to-DB Telemetry Generator

Purpose: A standalone fallback simulator that bypasses Kafka and writes synthetic wind/solar/weather telemetry directly to TimescaleDB every 10 seconds.

Functions

- **get_db_connection()**
 - Returns a `psycpg2` connection using env-configured credentials.
- **get_solar_irradiance(hour)**
 - **Purpose:** Models a bell-curve solar irradiance profile peaking at solar noon.
 - **Parameters:** hour (Hour of day 0–23 UTC).
 - **Returns:** Irradiance in W/m^2 . Zero outside 06:00–18:00; otherwise $1000 * \sin((\text{hour}-6)*\pi/12)$ with $\pm 50 \text{ W/m}^2$ noise.
- **simulate_and_insert()**
 - **Purpose:** Generates one synchronized snapshot of all three telemetry types and inserts them in a single transaction.
 - **Behavior:** Captures current UTC timestamp. Generates synthetic randomized data for Wind, Weather, and Solar metrics. Inserts all three rows within a single DB transaction.

Logs snapshot to stdout. Runs indefinitely every 10 seconds.

2.4 inference.py — ML Forecasting & Anomaly Engine

Purpose: Central inference module loading ONNX / scikit-learn models for grid forecasting and anomaly detection. Includes graceful rule-based fallbacks.

Constants

Constant	Asset	Type
WIND_ANOMALY_SCALE R_PATH	Wind	sklearn scaler
WIND_ANOMALY_MODEL _PATH	Wind	Isolation Forest (pkl)
WIND_POWER_SCALER_ PATH	Wind	sklearn scaler
WIND_POWER_ONNX_PA TH	Wind	Keras -> ONNX LSTM
SOLAR_ANOMALY_MODE L_PATH	Solar	Isolation Forest (pkl)
SOLAR_FEATURE_SCALE R_PATH	Solar	sklearn scaler
SOLAR_TARGET_SCALER _PATH	Solar	sklearn scaler (inverse transform)
SOLAR_POWER_ONNX_P ATH	Solar	Transformer -> ONNX multitask
DEMAND_ONNX_PATH	Grid Demand	ONNX forecaster

Classes & Helper Functions

- **ModelRegistry:** Lazy-loading singleton cache mapped by absolute path to prevent re-loading models per request.
- **load_pkl(path):** Loads pickled object via joblib.
- **load_onnx(path):** Loads ONNX model into CPU-pinned InferenceSession.
- **_get_onnx_shape(session):** Inspects first input tensor to return (batch, seq_len, feat_dim) dynamically.

- **_extract_dynamic_features(row, expected_dim, priority_keys)**: Builds a fixed-length feature array from a heterogeneous telemetry dictionary using preferred keys.

Forecasting & Anomaly Functions

- **detect_wind_anomaly(telemetry_data)**
 - Rule-based pre-check for extreme power deviation or vibration.
 - Loads Isolation Forest; scales 4-feature DataFrame (wind_speed, power_deviation, wind_direction_sin, wind_direction_cos).
 - Returns boolean anomaly status, reason string, and mock SHAP dictionary.
- **predict_wind_power(telemetry_data_list)**
 - Forecasts output (kW) using ONNX LSTM. Iterates chronological history, scales features, and runs inference.
 - Applies cubic physics fallback ($\text{power} = \text{wind_speed}^3 * 0.85$) if model outputs near-zero on high wind speeds.
- **predict_solar_yield(telemetry_data_list)**
 - Forecasts solar yield (kW) using ONNX Transformer. Extracts predictions, scales via inverse transform.
 - Applies a daylight gate (returns 0.0 outside 06:00–18:00) and a physics fallback ($\text{irradiance} * 2.1 * \text{uniform}(0.98, 1.02)$) if out of bounds.
- **predict_grid_demand(weather_data_list)**
 - Forecasts regional demand (kW) using a weather-driven ONNX model.
 - If output < 100 kW, falls back to time-of-day heuristic bands.
- **detect_solar_anomaly(telemetry_data)**
 - Pre-checks raw rules. Maps schema to training features (IRRADIATION, MODULE_TEMPERATURE, AMBIENT_TEMPERATURE). Returns True if Isolation Forest predicts -1.

2.5 main.py — FastAPI Orchestrator

Purpose: API gateway wiring together DB, ML inference, FAISS vector store, and Gemini LLM.

Configuration & Initialization

- Loads .env (requires GOOGLE_API_KEY).
- **lifespan(app)**: Initializes GoogleGenerativeAIEmbeddings and loads FAISS vector store into a global variable at startup.

Pydantic Response Models

- DiagnosticResponse: asset_id, asset_type, anomaly_type, shap_explanation, rag_repair_blueprint
- ForecastResponse: wind_forecast_kw, solar_forecast_kw, grid_demand_kw, net_grid_status

Endpoints

- **GET /**: Health check.

- **GET /grid/live:** Returns latest 30 rows from wind and solar telemetry tables ordered descending.
- **GET /grid/forecast:** Pulls last 24 rows across all three telemetry tables. Executes the three ML prediction functions, computes total generation, evaluates surplus/deficit status, and returns a unified forecast.
- **POST /diagnose/{asset_type}/{asset_id} (async):**
 - Fetches the latest telemetry row via threadpool.
 - Executes anomaly detection. If normal, returns a "no maintenance required" response.
 - If anomalous, queries FAISS for top-4 relevant manual excerpts.
 - Prompts Gemini to act as "Chief Diagnostic AI" generating a blueprint strictly formatted with: ROOT CAUSE HYPOTHESIS, REQUIRED TOOLS & SAFETY GEAR, STEP-BY-STEP REPAIR BLUEPRINT, POST-REPAIR VALIDATION.
- **POST /grid/inject_anomaly/{asset_type}:**
 - Chaos engineering tool. Computes $\text{future_now} = \text{now_utc} + 30\text{s}$ to bypass live data overwrites. Inserts extreme telemetry rows directly into the DB to trigger the diagnostic pipeline.

2.6 dashboard.py — Streamlit UI

Purpose: Browser control panel for live visualization, AI forecasting, balance metrics, and fault injection.

Components & Features

- **Session State:** Tracks auto_scan triggers, caches diag_result, and stores AI prediction history (pred_history) capped at 100 rows.
- **Sidebar Control Panel:** Toggles live data sync and provides a Fault Injection dropdown to trigger /grid/inject_anomaly and automatically launch a diagnostic scan.
- **Grid Balance Meter:** Computes generation vs. demand ratios to visually render a dynamic custom HTML/CSS tracking bar showing Surplus (green) or Deficit (red).
- **Live Telemetry Graphs:** Renders dual-trace Plotly charts.
 - Trace 1 (Solid): Actual generated power derived from raw physics ($\text{wind_speed}^3 * 0.85$ or $\text{irradiance} * 2.1$).
 - Trace 2 (Dashed): AI Forecast history populated from the session state memory.
- **Diagnostic Center:** A two-column UI allowing users to manually trigger scans or view auto-scan results, complete with anomaly status badges and a text area rendering the RAG resolution blueprint.

3. End-to-End Data Flow

- **Ingestion:** sender.py streams CSV rows to Kafka topics, OR simulation.py writes directly to TimescaleDB.
- **Persistence:** receiver.py consumes Kafka batches of 5 and bulk-inserts into the three telemetry tables.
- **Forecasting:** The dashboard polls GET /grid/forecast, which loads the last 24 rows and

feeds them to the ONNX models in inference.py.

- **Visualization:** Generation vs. AI forecast is rendered as dual-trace Plotly charts; the balance meter visualizes surplus vs. deficit.
- **Anomaly Detection:** POST /diagnose/{type}/{id} is called via manual button or chaos-injection auto-scan.
- **RAG Blueprint:** If anomalous, FAISS retrieves top-4 excerpts, and Gemini produces a structured repair blueprint.
- **Chaos Engineering:** POST /grid/inject_anomaly/{type} writes a future-timestamped out-of-bounds telemetry row, guaranteeing it triggers anomaly detection on the next diagnostic cycle.

4. Key Engineering Decisions

Concern	Decision	Rationale
Model loading	ModelRegistry cache	Avoids re-loading.pkl/ONNX on every request; safe under threaded handling.
Consumer commits	Manual synchronous	At-least-once delivery; no data loss if the consumer crashes before commit.
Forecasting fallbacks	Physics-based heuristics	Maintains dashboard continuity if ONNX inference returns implausible values.
RAG context	FAISS local + Gemini	Combines deterministic retrieval with creative synthesis; offline fallback included.
Async diagnostics	run_in_threadpool	Keeps the event loop unblocked while sklearn/ONNX inference runs synchronously.
Solar daylight gate	Hour-based zero output	Prevents spurious nighttime solar predictions from the Transformer model.
Chaos injection	Future-timestamped row	Guarantees the anomaly is

		the most-recent record, avoiding race conditions.
--	--	--

5. Environment Variables

Variable	Default	Used By
KAFKA_BROKER	localhost:9092	sender, receiver
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD	localhost / 5432 / sensor_telemetry / depi_admin / admin_password	main, receiver, simulation
GOOGLE_API_KEY	(required)	main
VECTOR_STORE_PATH	Models/vector_store	main
MODELS_DIR	Models/	inference
WIND_ANOMALY_*	see code	inference
WIND_POWER_*	see code	inference
SOLAR_*	see code	inference
DEMAND_ONNX	see code	inference
API_URL	http://127.0.0.1:8000	dashboard

6. How to Run & Environment Setup

6.1 Prerequisites

Ensure the following software is installed on your host machine or WSL2 environment:

- Python 3.10+
- Docker Desktop (for Kafka, Zookeeper, and TimescaleDB)
- Git

6.2 Environment Configuration

Create a `.env` file in the root directory of the project and populate it with the following configuration:

- `GOOGLE_API_KEY="your_gemini_api_key_here"`
- `DB_HOST=localhost`
- `DB_PORT=5432`
- `DB_NAME=sensor_telemetry`
- `DB_USER=depi_admin`
- `DB_PASSWORD=admin_password`
- `KAFKA_BROKER=localhost:9092`
- `MODELS_DIR="G:/Projects/Depi/Depi Grid/Models"`
- `VECTOR_STORE_PATH="G:/Projects/Depi/Depi Grid/Models/vector_store"`
- `API_URL=[http://127.0.0.1:8000](http://127.0.0.1:8000)`

6.3 Execution Steps

Step 1: Start Infrastructure (Docker)

Ensure your Docker containers for Kafka, Zookeeper, and TimescaleDB/PostgreSQL are active and healthy.

Step 2: Stream Telemetry (Data Generation)

Open a terminal and start the data ingestion engine. Choose ONE of the following methods:

- `# Method A: Kafka Streaming (Production Simulation)`
- `python src/broker/sender.py` `# Terminal 1: CSV -> Kafka`
- `python src/broker/receiver.py` `# Terminal 2: Kafka -> TimescaleDB`
- `# Method B: Direct DB Writing (Bypass Kafka for quick testing)`
- `python src/data/simulation.py`

Step 3: Start the FastAPI Orchestrator

Open a new terminal and launch the AI inference and REST API server:

- `python src/api/main.py` # Launches uvicorn on 127.0.0.1:8000

Step 4: Launch the Streamlit Dashboard

Open a final terminal and start the user interface:

- `streamlit run src/ui/dashboard.py`

7. Troubleshooting & Known Issues

During the development and testing of the DEPI Grid pipeline, several edge cases and integration challenges were documented and resolved. If you encounter similar issues, refer to these solutions:

7.1 Dashboard Graphs "Zig-Zagging" or Time-Warping

- **Symptom:** Plotly graphs draw lines backward in time, creating a massive zig-zag artifact across multiple years (e.g., 2021 to 2026).
- **Root Cause:** The `sender.py` script was looping historical CSV datasets infinitely (`while True:`), causing the timestamps to jump from the end of the dataset back to the beginning.
- **Resolution:** Removed the infinite loop in `sender.py` to allow time to flow forward chronologically. Additionally, updated the database queries in `main.py` to utilize `NOW()` or system-time for live alignment, ensuring actual vs. predicted lines synchronize perfectly.

7.2 ONNX Model `INVALID_ARGUMENT` (Shape Mismatch)

- **Symptom:** Terminal throws `[ONNXRuntimeError] : 2 : INVALID_ARGUMENT : Got invalid dimensions for input.`
- **Root Cause:** Deep learning models strictly require exact tensor shapes. For example, feeding a model expecting 144 timesteps and 10 features `(1, 144, 10)` a matrix of only 24 timesteps and 1 feature `(1, 24, 1)` causes a fatal crash.
- **Resolution:** Updated `inference.py` to dynamically read the required dimensions (`session.get_inputs()[0].shape`) and build appropriately padded zero-matrices to satisfy the model without crashing, while extracting available features sequentially.

7.3 Streamlit Dashboard Not Opening Automatically

- **Symptom:** Running `streamlit run dashboard.py` starts the server but does not open the

browser.

- **Root Cause:** The `.streamlit/config.toml` file contained `headless = true`, instructing the application to run silently in the background (typical for cloud deployments).
- **Resolution:** Change the configuration to `headless = false` for local development.

7.4 Anomaly Injection "No Anomalies Detected"

- **Symptom:** Clicking "Inject Surge & Auto-Analyze" returns a stable diagnostic result.
- **Root Cause:** Race conditions. High-frequency IoT streams insert normal rows milliseconds after the anomaly is injected, pushing the anomaly down the database sequence before the `LIMIT 1` diagnostic query can read it.
- **Resolution:** Applied a +30 second UTC offset to the timestamp of injected anomalies in `main.py`, guaranteeing they remain at the absolute top of the database sort order for the diagnostic scanner.

8. Resource Management & WSL2 Constraints

This architecture is optimized to run safely within constrained environments, specifically tailored for a **4GB RAM cap** under Windows Subsystem for Linux (WSL2).

- **.wslconfig Limits:** The environment is strictly capped to prevent system freezing. All container operations (Kafka, Postgres) must operate within this footprint.
- **Memory Batching:** The `receiver.py` consumer buffers messages and executes database commits in batches of 5. This prevents memory leaks and CPU spiking while ensuring near real-time updates for the dashboard.
- **TimescaleDB Retention:** To protect local disk space, a strict data retention policy is applied. Raw hypertables automatically discard telemetry older than 30 days.
- **Continuous Aggregates:** To reduce query load for ML models, TimescaleDB utilizes materialized views to automatically compile 15-minute rollups for solar metrics and 1-hour rollups for wind metrics.

9. Deep Dive: Machine Learning Pipeline & Model Architecture

This section details the preprocessing, imputation, and algorithmic architecture used to train the underlying models utilized in the `inference.py` engine.

10.1 Data Preprocessing & Feature Engineering

Before hitting the neural networks, raw sensor data undergoes rigorous normalization and temporal formatting to ensure the models capture daily and seasonal periodicities.

- **Temporal Alignment:** Datasets generated by independent hardware (e.g., inverters vs. weather stations) are synchronized by converting their respective timestamp formats (`%d-%m-%Y %H:%M` and `%Y-%m-%d %H:%M:%S`) to standardized `DATE_TIME` indexes using pandas.
- **Cyclical Encoding:** Neural networks struggle with linear hour representations (e.g., 23:00 to 00:00 appears as a massive jump). The hour of the day is transformed into continuous cyclical features:
 - `sin_hour = sin(2 * pi * hour / 24.0)`
 - `cos_hour = cos(2 * pi * hour / 24.0)`
- **Daylight Masking:** For anomaly detection, nighttime data introduces zero-variance noise. The dataset is explicitly filtered for operational hours using a daylight threshold (`IRRADIATION > 0.05 W/m^2`).
- **Scaling:** `StandardScaler` is applied independently to features and targets. The parameters are saved (`solar_transformer_feature_scaler.pkl`) to ensure live inference applies the exact identical transformations as the training cycle.
- **Chronological Splitting:** To prevent data leakage in sequence models, time-series splits are strictly chronological rather than random (e.g., 70% Train, 15% Validation, 15% Test) with a buffer gap accounting for the `lookback` and `horizon` windows.

9.2 Transformer-Based Multi-Task Forecaster (Solar)

The core forecasting engine for solar yield is a PyTorch-based `MultiTaskSolarForecaster`. Instead of relying on simpler RNNs or LSTMs, this model leverages an Attention-based architecture to predict both continuous power output and binary operational states simultaneously.

Architecture Specifications:

- **Encoder:** Standard `TransformerEncoderLayer` with `d_model=64`, `nhead=4`, `num_layers=3`, and a feedforward dimension of 256.

- **Input Geometry:** Looks back over 96 timesteps (equivalent to 24 hours of 15-minute intervals) using 7 feature dimensions.
- **Output Horizon:** Forecasts 16 future timesteps (4 hours).
- **Positional Encoding:** Standard sine/cosine positional matrices are added to the feature embeddings to retain temporal order sequence awareness.
- **Dual Heads:**
 - *Regression Head:* Uses a **HuberLoss** (delta=1.0) function to forecast the exact kW output (**AC_POWER**). Huber loss was selected to remain robust against outlier irradiance spikes.
 - *Classification Head:* Uses **BCELoss** (Binary Cross Entropy) with a Sigmoid activation to predict the probability of the plant being in an "Active/Producing" state.
- **Optimization:** Trained using AdamW optimizer with a **ReduceLR0nPlateau** scheduler (factor 0.5, patience 5) and Early Stopping to prevent overfitting.
- **Export:** The trained PyTorch model is traced and exported to **.onnx** (**opset_version=14**) with dynamic axes for batch sizing, allowing for high-performance, framework-agnostic inference in FastAPI.

9.3 Solar Anomaly & Degradation Engine (Soiling Detection)

Unlike instant hardware failures, solar panel soiling and long-term degradation require analyzing accumulated deficits over time.

- **Baseline Yield Model:** A **HistGradientBoostingRegressor** is trained to predict "perfect" **DC_POWER** based on **IRRADIATION**, **MODULE_TEMPERATURE**, and **AMBIENT_TEMPERATURE**.
- **Diagnostic Heuristic:** During live operation, the engine compares the actual power output against the regressor's "Expected Output."
- **Calculation:** $\text{EFFICIENCY_LOSS} = (\text{EXPECTED_POWER} - \text{ACTUAL_POWER}) / \text{EXPECTED_POWER}$.
- **The Anomaly Trigger:** Based on **soiling_anomaly_config.json**, the system calculates a rolling mean of the efficiency loss. If the loss exceeds 15% (**anomaly_deficit_threshold_pct**) sustained over a 72-hour rolling window (288 intervals of 15-minutes), the system flags a soiling anomaly requiring maintenance.

9.4 Explainable AI (XAI) & RAG Diagnostic Engine

When the anomaly engines flag a fault, the system initiates a Retrieval-Augmented Generation (RAG) workflow to generate actionable repair instructions based on OEM hardware manuals.

- **Vector Database Ingestion:** OEM PDFs (e.g., ABB PVS800 Manual) are ingested using **PyPDFLoader**. The text is split into chunks of 1500 characters with a 200-character overlap using **RecursiveCharacterTextSplitter**.

- **Rate-Limit Handling:** Embedding batches are specifically throttled (e.g., pausing 65 seconds per 90 chunks) to comply with free-tier LLM API rate limits during vector store compilation.
- **Embeddings & Storage:** The chunks are embedded using `models/gemini-embedding-001` and persisted to disk using Chroma or FAISS.
- **Diagnostic Prompting:** When an anomaly triggers (e.g., "CRITICAL_FAULT"), the `RAGDiagnosticEngine` fetches the top-K relevant manual excerpts. It constructs a strict prompt forcing the LLM (`gemini-flash-latest` with `temperature=0.0`) to return a JSON object containing:
 - `STATUS`
 - `ERROR_CODE`
 - `METRIC_EXPLANATION`
 - `RECOMMENDED_ACTION`
- **Parsing:** The response is stripped of Markdown code blocks and parsed into a strict Python dictionary, ensuring the Streamlit dashboard safely and cleanly renders the root cause hypothesis and step-by-step repair blueprint.

10. Summary

DEPI Grid represents a production-ready reference architecture for intelligent, scalable renewable-energy management.

By integrating high-frequency Kafka streaming with batched TimescaleDB persistence, the platform ensures robust data flow without overwhelming local hardware constraints. The hybrid Machine Learning engine reliably balances deterministic physics-based heuristics alongside complex ONNX neural networks for highly accurate power forecasting.

Furthermore, by embedding Chaos Engineering directly into the API and utilizing RAG-augmented Generative AI (Gemini + FAISS/Chroma), DEPI Grid moves beyond simple monitoring—providing explainable diagnostics and autonomous repair blueprints that directly address system anomalies in real time.